# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

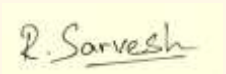
## CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1021 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Scenario-Based Requirement Analysis |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | R SARVESH | **Reg. No.:** | 192521143 |
| **Date:** | 30/10/2026 | **Duration:** | 60 minutes |

### Code of Conduct

*I R SARVESH / 192521143 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



## Annexure A – Scenario-Based Requirement Analysis

### Rubrics for Calculation

| Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Scenario Understandin g and Problem Analysis (30%)** | Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis. | Demonstrates good understanding with minor omissions. | Basic understanding with limited analysis. | Poor understanding; major aspects missing or incorrect. |
| **Functional and Non-Functional Requirement Identification (25%)** | Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements. | Identifies most requirements with minor classification errors. | Identifies only basic requirements; several omissions. | Requirements are incomplete, inaccurate, or poorly classified. |
| **Actor Identification and Use Case Modelling (20%)** | Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards. | Most actors and use cases identified; diagram has minor errors. | Limited actors/use cases identified; diagram partially correct. | Actors/use cases missing or incorrect; diagram absent or inaccurate. |

| | | | | |
|---|---|---|---|---|
| **Software Requirement Specification (SRS) Documentati on (15%)** | SRS is well-structured, complete, professionally organized, and includes all required sections. | SRS is organized with minor missing sections or formatting issues. | SRS includes only essential sections with limited detail. | SRS is incomplete, poorly organized, or lacks essential components. |
| **Assumptions, Constraints, and Requirement Validation (10%)** | Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification. | Identifies most assumptions and constraints; validation is adequate. | Limited assumptions/c onstraints; validation is superficial. | Assumptions, constraints, and validation are missing or incorrect. |

| Criteria | **Mark** |
|---|---|
| Scenario Understanding and Problem Analysis (30%) | /5 |
| Functional and Non-Functional Requirement Identification (25%) | /5 |
| Actor Identification and Use Case Modelling (20%) | /5 |
| Software Requirement Specification (SRS) Documentation (15%) | /5 |
| Assumptions, Constraints, and Requirement Validation (10%) | /5 |
| **TOTAL** | /25 |

# 1. Autonomous Drone-Carrier Mesh Network for Hyper-Local Medical Logistics

## Industry Context

Modern municipal emergency response infrastructures are heavily bottlenecked by urban traffic congestion, grid design limits, and unpredictable weather anomalies. During critical medical crises—such as sudden cardiac arrests, acute anaphylaxis, or postpartum hemorrhages—the delivery time of life-saving medical supplies (e.g., Automated External Defibrillators (AEDs), EpiPens, rare blood types, or anti-venoms) determines patient survival. Traditional ambulance response times in dense metropolises often stretch beyond the critical 4-to-6-minute survival window.

While commercial drone delivery networks are emerging, individual drone units suffer from limited operational ranges, rapid battery depletion, and high vulnerability to sudden micro-climate shifts. Centralized cloud-based dispatch systems introduce single points of failure and high network latency penalties, which are unacceptable in life-or-death scenarios. There is an immediate structural demand for a decentralized, edge-computed logistics platform that treats automated delivery vehicles, municipal rooftops, and public transit nodes as a cohesive, self-healing transit mesh network.

## Problem Statement

Design and develop a highly concurrent, fault-tolerant Autonomous Drone-Carrier Mesh Network that dynamically routes, schedules, and dispatches automated medical delivery drones across dynamic smart city topographies.

The platform must ingest high-velocity streaming telemetry—including real-time 3D drone coordinates, localized wind velocity vectors, active airspace geofences, battery health diagnostics, and regional cellular signal strengths—from hundreds of airborne units and municipal charging hubs concurrently.

Using decentralized routing engines, the platform must coordinate drone handovers between automated charging platforms mounted on moving public transit buses and static municipal smart buildings. The solution must run autonomous collision avoidance, handle sudden edge computing node dropouts, and guarantee zero-latency emergency rerouting without relying on continuous, active cloud infrastructure connectivity.

## Objectives

The platform should:

- **Ingest and Process Sub-Second Telemetry:** Capture high-frequency 3D spatial positions and hardware diagnostic data from hundreds of active airborne drone arrays without packet loss.
- **Orchestrate Dynamic 4D Routing:** Compute optimal, collision-free 4D flight corridors (latitude, longitude, altitude, time) that adapt dynamically to shifting wind patterns and temporary no-fly zones.
- **Manage Moving Node Handovers:** Coordinate micro-second landing and deployment schedules onto mobile public transit carrier platforms using localized edge mesh communication loops.
- **Enforce Edge Autonomy & Self-Healing:** Implement decentralized routing protocols that allow the local swarm to self-heal and re-allocate tasks autonomously if the central cloud goes offline.
- **Ensure Mission-Critical Prioritization:** Design a strict traffic prioritization engine that automatically clears airspace and throttles commercial data streams when a high-priority medical alert is triggered.
- **Generate Cryptographic Provenance Audits:** Maintain unalterable, cryptographically signed operational logs to satisfy civil aviation authorities and medical privacy compliance boards.

## Expected Outcomes

The proposed platform will:

- **Minimize Life-Saving Delivery Windows:** Reduce the average municipal transit window for critical medical payloads to under 4 minutes.
- **Extend Fleet Operational Reach:** Increase effective drone flight ranges by up to 300% through optimized, multi-stage mobile carrier handovers.
- **Guarantee High System Availability:** Achieve 99.999% network uptime for emergency dispatch tracking via fully decentralized edge computing worker networks.
- **Lower Operational Asset Losses:** Eliminate mid-air drone collisions and forced emergency landings via highly responsive, predictive collision avoidance networks.
- **Provide Regulatory Transparency:** Enable seamless compliance tracking for aviation authorities through instantly generated, immutable flight performance audit trails.

**Software Requirements Specification (SRS)**
Autonomous Drone-Carrier Mesh Network for Hyper-Local Medical Logistics

**Project Type:** Scenario-Based Requirement Analysis
**Domain:** Emergency Medical Logistics, Unmanned Aerial Systems (UAS), Edge Computing, Distributed Real-Time Systems

## 1. Study and Analyze the Scenario

### 1.1 Introduction

The Autonomous Drone-Carrier Mesh Network is a decentralized, edge-computed logistics platform for delivering life-saving medical supplies (AEDs, EpiPens, rare blood types, anti-venoms) inside dense urban areas. During cardiac arrest, anaphylaxis or postpartum hemorrhage, survival depends on delivery within a 4-to-6-minute window. Ambulances in congested cities often miss this window.

A single drone has limited range, depletes its battery quickly and is vulnerable to sudden wind or weather changes. Centralized cloud dispatch adds latency and a single point of failure. The proposed system therefore treats drones, charging platforms on moving public-transit buses, and charging platforms on static municipal buildings as one self-healing mesh network.

The system:

- ingests high-frequency telemetry from hundreds of drones and charging hubs
- computes collision-free 4D (latitude, longitude, altitude, time) flight corridors
- coordinates drone handovers between moving and static carrier platforms
- keeps operating through edge-node dropouts and cloud outages
- gives medical emergencies absolute priority over commercial traffic
- records cryptographically signed, tamper-evident audit logs for aviation and medical-privacy compliance

### 1.2 Business Domain

- Emergency medical services (EMS) and smart-city infrastructure
- Unmanned aerial vehicle (UAV) traffic management and logistics
- Edge and distributed computing
- Real-time streaming telemetry processing
- Public transit and municipal asset integration
- Aviation regulatory compliance and medical data privacy

## 1.3 Stakeholders

| **Stakeholder** | **Role** |
|---|---|
| Patients / emergency requesters | Receive life-saving supplies; their survival depends on delivery time |
| Emergency dispatchers / medical operators | Trigger and track high-priority medical alerts |
| Fleet operators | Monitor drone health, mission status and fleet utilization |
| Public transit authority | Hosts charging/landing platforms on buses; defines operating constraints |
| Municipal building owners | Host static rooftop charging hubs |
| Civil aviation authority | Regulates airspace, geofences and flight compliance |
| Medical privacy / compliance boards | Ensure payload and patient data handling is lawful |
| | Supply the medical payloads |
| System administrators | Manage nodes, users, configuration and availability |
| Weather and cellular data providers | Supply wind, weather and signal-strength data |
| Commercial drone / data-stream users | Share airspace and bandwidth; are throttled during emergencies |
| Developers / maintenance team | Build, deploy and maintain the platform |

## 1.4 Existing Challenges

1. Urban traffic congestion pushes ambulance response beyond the 4-to-6-minute survival window.
2. Single drones have short range, rapid battery depletion and no way to extend flights.
3. Sudden wind shifts and micro-climate changes make flight paths unsafe.
4. Centralized cloud dispatch adds network latency and a single point of failure.
5. Hundreds of drones sharing airspace create mid-air collision risk.
6. Moving carrier platforms (buses) make landing and takeoff scheduling hard.
7. Temporary no-fly zones and geofences change dynamically.
8. Commercial data traffic can compete with emergency traffic for bandwidth and airspace.
9. Cellular signal strength varies across the city, causing packet loss and communication blind spots.
10. Regulators and privacy boards require immutable proof of operations.

### 1.5 System Objectives

- Ingest and process sub-second telemetry from hundreds of drones without unrecovered data loss.
- Compute optimal, collision-free 4D flight corridors that adapt to wind and no-fly zones.
- Coordinate precise landing and deployment handovers on mobile and static carrier platforms.
- Operate autonomously at the edge and self-heal when the cloud or edge nodes fail.
- Enforce strict emergency prioritization, clearing airspace and throttling commercial traffic.
- Maintain signed, tamper-evident audit logs.
- Support the expected outcomes: delivery in under 4 minutes, up to 300% extended range, 99.999% emergency-dispatch availability, no mid-air collisions, and instant audit trails.

## 2. Functional and Non-Functional Requirements

### 2.1 Functional Requirements

| ID | Requirement |
|---|---|
| FR01 | The system shall ingest real-time telemetry (3D position, wind vectors, battery diagnostics, signal strength) from all active drones. |
| FR02 | The system shall ingest status data from municipal charging hubs and bus-mounted carrier platforms. |
| FR03 | The system shall validate, timestamp and sequence-number all incoming telemetry, and detect gaps, duplicates and stale messages. |
| FR04 | The system shall maintain a live airspace model including active geofences, no-fly zones and weather data. |
| FR05 | The system shall compute collision-free 4D flight corridors (latitude, longitude, altitude, time). |
| FR06 | The system shall recompute routes dynamically when wind, geofences or drone health change. |
| FR07 | The system shall run predictive, autonomous collision avoidance and issue avoidance maneuvers. |
| FR08 | The system shall select the best available drone and delivery route for each medical request, based on payload, distance, battery and carrier availability. |
| FR09 | The system shall plan multi-stage routes with intermediate handovers to extend effective range. |
| FR10 | The system shall schedule and coordinate landing and takeoff on moving bus platforms and static building hubs using local edge mesh communication. |
| FR11 | The system shall let authorized dispatchers raise a high-priority medical alert and track the delivery. |

| | |
|---|---|
| FR12 | On a high-priority alert, the system shall clear conflicting airspace, preempt lower-priority missions and throttle commercial data streams. |
| FR13 | The system shall let the local swarm operate and re-allocate tasks autonomously when the cloud is unavailable. |
| FR14 | The system shall detect edge-node dropouts and migrate their responsibilities to neighboring nodes. |
| FR15 | The system shall trigger safe fallback behavior (return-to-hub, nearest safe landing, hold) on communication loss, low battery or hazard. |
| FR16 | The system shall provide a real-time dashboard for fleet status, airspace, deliveries and alerts. |
| FR17 | The system shall let authorized users manage geofences and no-fly zones. |
| FR18 | The system shall generate cryptographically signed, hash-chained, append-only logs of flight, routing, handover and prioritization decisions. |
| FR19 | The system shall give auditors and regulators read-only access to audit trails and generate compliance reports on demand. |
| FR20 | The system shall protect patient-related and payload data in line with medical-privacy rules. |
| FR21 | The system shall provide role-based access control and authenticate all users, drones and edge nodes. |
| FR22 | The system shall synchronize local edge state and logs with the cloud when connectivity returns and resolve conflicts deterministically. |

## 2.2 Non-Functional Requirements

| **ID** | **Category** | **Requirement / Acceptance Criterion** |
|---|---|---|
| NFR01 | Performance (telemetry) | Edge ingestion and normalization latency ≤ 100 ms (99th percentile) under a defined benchmark of 500 drones at 10 Hz each. |
| NFR02 | Performance (response) | Emergency re-routing and collision-avoidance decisions at the edge within a bounded target (e.g., ≤ 200 ms), measured separately from physical flight response. |
| NFR03 | Mission performance | Target average delivery window < 4 minutes for critical payloads within the service area, verified by simulation and pilot trials. |
| NFR04 | Scalability | Support at least 500 concurrent drones and 100 hubs, scaling horizontally by adding edge nodes. |

| | | |
|---|---|---|
| NFR05 | Availability | 99.999% availability for emergency dispatch tracking (about 5.26 minutes of downtime per year), with the measurement method defined. |
| NFR06 | Fault tolerance / self-healing | Detect node failure and reassign its tasks within a defined target (e.g., ≤ 2 seconds) with no loss of in-flight mission state. |
| NFR07 | Reliability | No unrecovered telemetry loss: sequence gaps are detected and retransmitted or flagged. |
| NFR08 | Safety | Maintain minimum separation distances between drones; deliver zero mid-air collisions in simulation and certified testing. |
| NFR09 | Autonomy | Core dispatch, routing and collision avoidance shall run fully offline from the cloud. |
| NFR10 | Security | mTLS between all nodes, encrypted data at rest and in transit, signed firmware and commands, and protection against spoofing and jamming. |
| NFR11 | Integrity / Auditability | Tamper-evident logs (hash-chained, digitally signed); retention per aviation and privacy policy. |
| NFR12 | Privacy / Compliance | Comply with applicable aviation regulations and medical data protection law. |
| NFR13 | Interoperability | Open, versioned APIs (OpenAPI) for integration with EMS, transit and airspace systems. |
| NFR14 | Usability | The dashboard shall allow a dispatcher to raise an emergency alert in ≤ 3 interactions. |
| NFR15 | Maintainability | Modular, containerized services with automated tests and version-controlled configuration. |
| NFR16 | Energy efficiency | Routing shall minimize energy use while respecting battery reserve margins. |

## 3. Actors and Use Cases

### 3.1 Primary and Secondary Actors

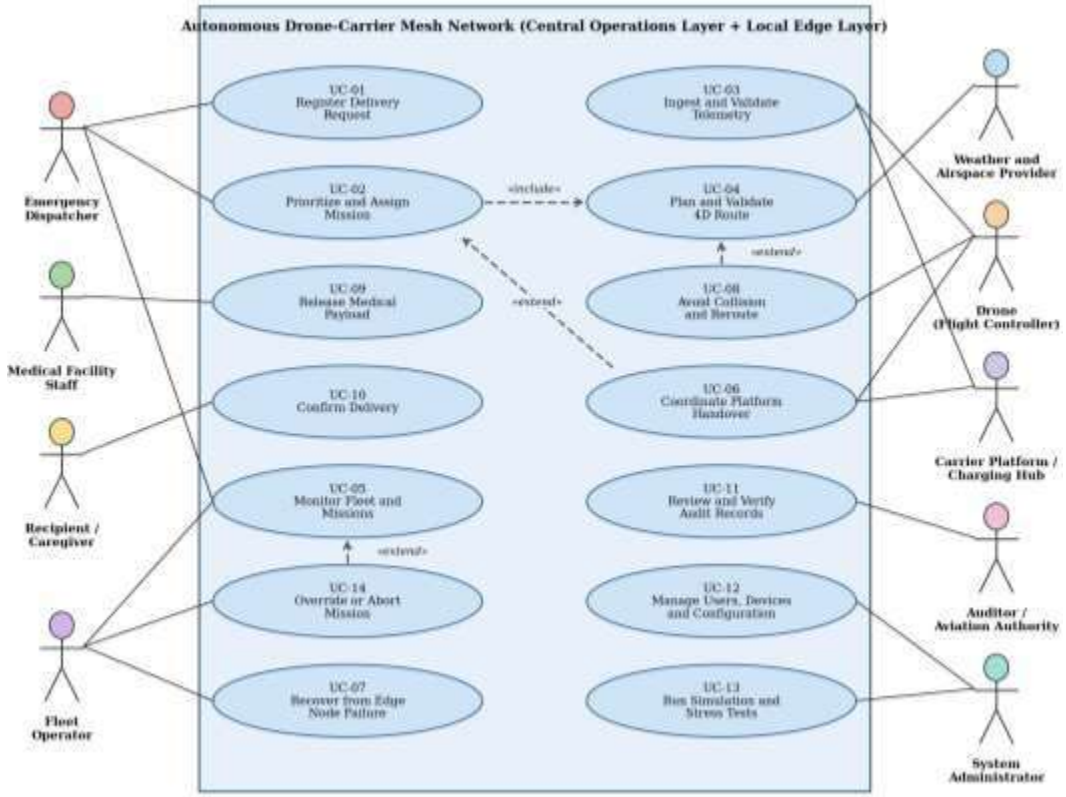
| **Actor** | **Type** | **Responsibilities** |
|---|---|---|
| Emergency Dispatcher | Primary | Raises high-priority medical alerts and tracks deliveries. |
| Fleet Operator | Primary | Monitors drone and hub health, and manages geofences. |
| System Administrator | Primary | Manages users, nodes, configuration and system health. |

| | | |
|---|---|---|
| Auditor / Aviation Regulator | Primary | Reviews signed logs and compliance reports. |
| Drone Unit | Secondary | Sends telemetry and executes flight and handover commands. |
| Carrier Platform Controller (bus / building hub) | Secondary | Reports position, charging status and landing readiness. |
| Weather / Wind Data Service | Secondary | Supplies wind and weather data. |
| Airspace / Geofence Authority System | Secondary | Supplies no-fly zones and airspace restrictions. |
| Cloud Backend | Secondary | Provides long-term storage, analytics and synchronization (optional at run time). |

## 3.2 Major Use Cases

| **ID** | **Use Case** | **Actor** |
|---|---|---|
| UC-01 | Ingest drone and hub telemetry | Drone Unit, Carrier Platform Controller |
| UC-02 | Raise emergency medical alert | Emergency Dispatcher |
| UC-03 | Plan 4D flight corridor | Routing Engine |
| UC-04 | Perform collision avoidance | Routing Engine, Drone Unit |
| UC-05 | Coordinate carrier handover | Carrier Platform Controller, Drone Unit |
| UC-06 | Prioritize emergency traffic | Priority Engine |
| UC-07 | Self-heal on node failure | Edge Mesh |
| UC-08 | Operate in cloud-offline mode | Edge Mesh |
| UC-09 | Monitor fleet dashboard | Fleet Operator, Dispatcher |
| UC-10 | Manage geofences / no-fly zones | Fleet Operator, Airspace Authority |
| UC-11 | Manage users and configuration | System Administrator |
| UC-12 | Generate signed audit log | Audit Service |
| UC-13 | Review audit and compliance records | Auditor |
| UC-14 | Synchronize edge state with cloud | Cloud Backend |

## 3.3 Use Case Diagram




## 3.4 Use Case Description: Raise Emergency Medical Alert and Dispatch

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-02 |
| Use Case Name | Raise Emergency Medical Alert and Dispatch |
| Primary Actor | Emergency Dispatcher |
| Supporting Actors | Drone Unit, Carrier Platform Controller, Weather/Geofence Services |
| Precondition | The dispatcher is authenticated, at least one drone and payload are available, and the edge mesh is operational. |
| Trigger | A critical medical emergency (e.g., cardiac arrest) is reported. |

| | |
|---|---|
| Main Flow | 1. Dispatcher enters the emergency type, location and payload. 2. System validates the request and tags it as top priority. 3. Priority Engine clears conflicting airspace and throttles commercial streams. 4. System selects the best drone by distance, battery and payload fit. 5. Routing Engine computes a collision-free 4D corridor using current wind and geofences. 6. Multi-stage handovers are planned if the range is insufficient. 7. Drone is dispatched and tracked in real time. 8. Payload is delivered and confirmed. 9. All decisions are written to the signed audit log. |
| Alternative Flow | A1: If no drone has enough range, plan a handover via a bus or building hub. A2: If the cloud is offline, the local edge mesh makes all decisions. A3: If the assigned drone fails, immediately reassign to the next best drone. A4: If wind or a geofence blocks the route, recompute the corridor or re-dispatch. |
| Postcondition | The payload is delivered, the mission outcome is recorded, and an immutable audit entry exists. |

## 3.5 Use Case Description: Coordinate Moving Carrier Handover

| **Field** | **Description** |
|---|---|
| Use Case ID | UC-05 |
| Primary Actor | Carrier Platform Controller |
| Supporting Actors | Drone Unit, Routing Engine |
| Precondition | A multi-stage route requires a handover to a bus-mounted or building-mounted platform with available capacity. |
| Trigger | The drone approaches the planned handover point or a battery threshold is reached. |
| Main Flow | 1. Routing Engine predicts the bus position and time at the rendezvous. 2. Carrier controller confirms platform readiness and charging slot. 3. Drone and carrier exchange position and velocity over the local mesh. 4. System computes a landing window and approach path. 5. Drone lands and the payload is secured or recharged. 6. Drone redeploys with a fresh battery or the payload transfers to another drone. 7. Handover result is logged. |
| Alternative Flow | If the platform is unavailable or the bus deviates, select an alternate platform or the nearest static hub. If the landing window is missed, replan the rendezvous. |
| Postcondition | The drone continues its mission or is safely parked, and the handover is recorded in the audit log. |

## 4. Software Requirement Specification (SRS)

### 4.1 Introduction

This SRS defines the functional capabilities, quality requirements, interfaces, data and operational constraints of the Autonomous Drone-Carrier Mesh Network. It is the reference for design, implementation, testing, regulatory review and acceptance.

### 4.2 Problem Description

Urban emergency response is limited by congestion, grid layout and weather. Individual drones have short range and are vulnerable to weather, while cloud-centric dispatch adds latency and single points of failure. A resilient, decentralized platform is needed that continuously ingests telemetry, plans collision-free 4D routes, hands drones over between moving and static carriers, and keeps working without the cloud, all while prioritizing medical emergencies and providing regulator-grade audit evidence.

### 4.3 Scope of the System

**In scope:**

- Telemetry ingestion, validation and airspace modeling
- 4D route planning and predictive collision avoidance
- Moving and static carrier handover coordination
- Emergency prioritization and commercial traffic throttling
- Edge autonomy, self-healing and cloud synchronization
- Dashboards, geofence management and role-based access
- Cryptographically signed audit logging and compliance reporting

**Outside the initial scope:**

- Drone hardware, airframe and battery manufacturing
- Medical diagnosis or clinical decision-making
- Guaranteeing zero accidents or flight in all weather
- Replacing the national air traffic control system
- Guaranteeing cellular or public-network performance

## 4.4 System Features

| Feature | Description |
|---|---|
| Telemetry Ingestion Pipeline | Receives, validates and sequences high-frequency data from drones and hubs. |
| 4D Routing Engine | Computes collision-free corridors adapted to wind and no-fly zones. |
| Collision Avoidance | Predicts conflicts and issues separation maneuvers. |
| Carrier Handover Manager | Schedules landings and deployments on moving and static platforms. |
| Priority Engine | Preempts traffic and throttles commercial streams for emergencies. |
| Edge Mesh and Self-Healing | Runs decentralized consensus, failover and task reallocation. |
| Audit and Provenance Service | Produces signed, hash-chained, immutable logs. |
| Dashboard and Alerting | Gives dispatchers and operators live visibility and control. |

## 4.5 Data Requirements

| Data Entity | Important Attributes | Purpose |
|---|---|---|
| Drone Telemetry | Drone ID, lat, lon, alt, velocity, battery, health, signal, sequence no., timestamp | Tracking and routing |
| Carrier Platform | Platform ID, type (bus/building), position, speed, slot availability, charge level | Handover planning |
| Airspace / Geofence | Zone ID, polygon, altitude range, validity period, source | Route restriction |
| Weather / Wind | Region, wind vector, gust, timestamp | Route adaptation |
| Mission | Mission ID, priority, payload type, origin, destination, assigned drone, status | Dispatch tracking |
| Flight Corridor | Corridor ID, 4D waypoints, validity window | Collision-free routing |
| Handover Event | Event ID, drone, platform, window, result | Handover audit |
| Edge Node | Node ID, role, health, last heartbeat | Self-healing |
| Audit Record | Record ID, event, timestamp, previous hash, signature | Provenance |
| User | User ID, role, permissions | Access control |

## 4.6 External Interface Requirements

- **User interface:** web and mobile dashboard for dispatchers, operators, administrators and auditors.
- **Software interfaces:** REST/gRPC APIs and event streams for EMS dispatch, transit systems, weather services and airspace authorities.
- **Hardware interfaces:** drone flight controllers, GPS/IMU sensors, carrier-platform charging and landing hardware, and edge computing nodes.
- **Communication interfaces:** low-latency mesh messaging (e.g., MQTT/AMQP/gRPC) with mTLS, plus cellular and short-range radio links.
- **Audit interface:** read-only, signed export APIs for regulators.

## 4.7 Requirement Prioritization (MoSCoW)

| **Priority** | **Requirements** |
|---|---|
| **Must Have** | Telemetry ingestion and validation (FR01–FR03); 4D routing and collision avoidance (FR05–FR07); emergency alert and prioritization (FR11–FR12); edge autonomy and failover (FR13–FR15); signed audit logging (FR18); authentication and access control (FR21) |
| **Should Have** | Multi-stage handover range extension (FR09–FR10); geofence management UI (FR17); cloud synchronization (FR22); compliance reporting (FR19) |
| **Could Have** | Predictive battery-health analytics; energy-optimized routing (NFR16); mobile dispatcher app enhancements |
| **Won't Have (for now)** | Fully autonomous clinical triage; cross-city federation; heavy-cargo drone support |

## 4.8 Kano Classification

| **Category** | **Features** |
|---|---|
| **Must-Be (Basic)** | Collision avoidance, secure authentication, reliable emergency dispatch, regulatory-compliant logging |
| **Performance** | Delivery time, routing speed, system availability, telemetry latency |

| | |
|---|---|
| **Exciting (Delighters)** | Predictive wind-aware rerouting, automatic range extension through bus handover, live ETA sharing with responders |
| **Indifferent** | Decorative dashboard themes, non-essential animations (candidates for removal) |

## 4.9 Elicitation Approach

- **Focus groups** with dispatchers, fleet operators and transit staff to explore workflow needs early. A trained facilitator guards against groupthink.
- **Prototypes** (low-fidelity wireframes, then clickable mockups) of the dispatch dashboard, to validate ambiguous visual workflows. Stakeholders must understand that a prototype is not a finished product.
- **Document analysis** of aviation regulations and medical-privacy rules, **observation** of ambulance dispatch operations, and **interviews** with aviation authority representatives.

## 4.10 Architecture Considerations

| Pattern | Application in this System |
|---|---|
| **Event-driven architecture** | Telemetry, alerts and handover events flow through a message broker (e.g., Kafka/MQTT). Consumers are decoupled, and events queue safely if a consumer fails. |
| **Microservices** | Routing, Priority, Handover, Audit and Dashboard are independent, separately deployable, horizontally scalable services. |
| **Distributed systems** | Edge nodes are replicated across the city with consensus (e.g., Raft) for leader election and state replication. Network partitions and eventual consistency are accepted trade-offs. |
| **Serverless (limited)** | Used only for non-critical cloud tasks such as report generation, not for latency-critical edge paths. |
| **Containerization** | Docker images for all services; Kubernetes (or lightweight K3s at the edge) for scheduling, self-healing and auto-scaling. |
| **API-first** | OpenAPI contracts are finalized before implementation so frontend, edge and third-party teams can work in parallel against mocks. An API gateway handles authentication and rate limiting. |
| **API versioning** | URL path versioning (/api/v1/...) with backward compatibility within a major version. |

## 4.11 Design by Contract and Formal Verification

**Design by Contract :**

| Element | Contract |
|---|---|
| Precondition | The drone is healthy, battery ≥ required reserve, the corridor is inside no active geofence, and the corridor is conflict-free for its time window. |
| Postcondition | The corridor is reserved exclusively for that time window, and the reservation is logged. |
| Invariant | No two reserved corridors ever violate the minimum separation distance. |

**Formal verification:** Use model checking (e.g., TLA+ or SPIN) to verify the handover protocol and the airspace-reservation algorithm for all possible states. The goals are no collision state, no deadlock during handover, and no loss of mission state during failover. Contracts also give verification tools a precise specification to work from.

## 4.12 Requirement Conflict Resolution and Collaborative Documentation

- Track each requirement as a Jira work item with acceptance criteria and a peer-review workflow.
- Maintain the SRS in Confluence with status labels (Draft / In Review / Verified), inline comments and @mentions, and page-history rollback.
- Link Confluence requirement pages to Jira epics and user stories to keep a single source of truth.
- Resolve deadlocks with the **DACI** framework (Driver, Approver, Contributor, Informed) and record each decision as an Architectural Decision Record (ADR).
- **Example conflict:** *Safety* (reject any route lacking a full battery reserve) versus *speed* (accept the shortest route in an emergency). It is resolved by an approved emergency battery-reserve rule decided under DACI, with the approver being the safety/aviation lead.

## 5. Assumptions and Constraints

### 5.1 Assumptions

1. Drones expose a documented telemetry interface and support command acknowledgment.
2. Public transit authorities permit mounting charging platforms on buses and publish reliable route and schedule data.
3. Municipal buildings provide rooftop space, power and network access for hubs.
4. Weather, wind and geofence data services are available in a documented format.
5. Edge nodes have enough computing power and local storage for autonomous operation.
6. Aviation authorities approve beyond-visual-line-of-sight (BVLOS) emergency operations.
7. Medical payloads are prepared and handed to drones by authorized personnel.
8. Cellular and mesh radio coverage is sufficient in most of the service area, with fallbacks for dead zones.
9. Risk models and thresholds are reviewed and approved before deployment.

### 5.2 Constraints

| **Constraint Type** | **Description** |
|---|---|
| Regulatory | Compliance with civil aviation rules (airspace, altitude, BVLOS, geofencing) and medical-privacy law. |
| Technical | Edge hardware limits, battery capacity, payload weight and drone range. |
| Environmental | Wind, rain, fog and temperature limit safe flight. |
| Real-time | Hard timing bounds for collision avoidance and emergency rerouting. |
| Safety | Mandatory minimum separation and battery-reserve rules cannot be overridden. |
| Security | Strong authentication, encrypted communication and spoof/jamming resistance. |
| Economic | Costs of drones, carrier platforms, edge nodes, connectivity and certification. |
| Time-related | Development, testing and regulatory approval must fit the project schedule. |
| External dependency | Cellular networks, GPS accuracy and third-party data feeds are outside the system's direct control. |

## 6. Requirement Validation

### 6.1 Completeness and Consistency Checklist

| Validation Item | Verification Method | Expected Result |
|---|---|---|
| Stakeholder coverage | Map each stakeholder need to an FR or NFR | All major needs are addressed |
| Functional completeness | Walk through ingestion, routing, handover, prioritization, audit | No major workflow missing |
| Non-functional measurability | Define a test for every quality attribute | Each NFR has an acceptance criterion |
| Ambiguity | Review terms such as "zero latency", "no packet loss", "optimal" | Replaced with measurable targets |
| Redundancy | Compare overlapping requirements | Each requirement has a distinct purpose |
| Consistency | Check priority rules against safety rules | No conflicting processing rules |
| Traceability | Link requirements to use cases and test cases | Every requirement is verifiable |
| Safety and recovery | Test node failure, cloud outage, signal loss, low battery | Safe-response behavior is defined |
| Feasibility | Review with aviation and engineering experts | Requirements are technically achievable |

### 6.2 Improvements Recommended

1. **Define "latency" precisely:** specify whether a target measures ingestion, routing computation, command delivery or physical response.
2. **Replace absolute claims:** convert "zero-latency", "without packet loss" and "eliminate collisions" into measurable, risk-based targets.
3. **Clarify "micro-second landing":** physical landings cannot be scheduled in microseconds. Define the timing granularity of the handover protocol (e.g., millisecond-level coordination and a seconds-level landing window).
4. **Define the availability method:** state how the 99.999% figure is measured, over which components and with which exclusions.

5. **Define the range-extension baseline:** state the reference drone and conditions behind the "300%" figure.
6. **Specify degraded modes:** define behavior for GPS loss, jamming, partitioned networks and conflicting leaders.
7. **Define conflict-resolution rules:** specify deterministic ordering for simultaneous emergencies competing for the same corridor or platform.
8. **Add audit policy:** define retention periods, key management and access rules.
9. **Test extreme scenarios:** include simultaneous emergencies, sudden storms, bus breakdowns, mass node failure and a cloud outage during peak load.

## 6.3 Requirement Traceability Matrix

| **Objective** | **Related Requirements** | **Verification** |
|---|---|---|
| Ingest sub-second telemetry | FR01–FR03, NFR01, NFR07 | Load and latency tests, gap-detection tests |
| Dynamic 4D routing | FR04–FR06, FR08 | Routing simulation with wind and geofence changes |
| Collision avoidance | FR07, NFR02, NFR08 | Multi-drone conflict simulations, formal model checking |
| Moving-node handover | FR09–FR10 | Handover protocol simulation and field trials |
| Edge autonomy and self-healing | FR13–FR15, FR22, NFR05, NFR06, NFR09 | Chaos testing (node kill, cloud disconnect) |
| Mission-critical prioritization | FR11–FR12 | Priority preemption tests under load |
| Cryptographic provenance | FR18–FR19, NFR11 | Tamper-detection and signature-verification tests |
| Security and privacy | FR20–FR21, NFR10, NFR12 | Penetration and compliance audits |
| Delivery-time outcome | NFR03 | End-to-end simulation and pilot measurement |

## 7. Conclusion

This SRS defines the requirements for the Autonomous Drone-Carrier Mesh Network for hyper-local medical logistics. It covers stakeholders, functional and non-functional requirements, actors, use cases, data structures, interfaces, prioritization (MoSCoW and Kano), architectural patterns, design-by-contract specifications, assumptions, constraints and validation. The system aims to cut emergency delivery times, extend drone reach through carrier handovers, stay available without the cloud, avoid collisions, and give regulators verifiable evidence of its operations.

Before implementation, the latency, availability, safety and range-extension targets must be converted into measurable acceptance tests and verified through simulation, formal verification, field trials and regulatory certification. This will help ensure the platform is safe, reliable, scalable and suitable for life-critical urban deployment.